KỸ THUẬT LẬP TRÌNH / JAVA

CHƯƠNG 1: INTRODUCTION TO JAVA PROGRAMMING

Buổi học tích hợp Lý thuyết & Thực hành tại lớp (Thời lượng: 3 Tiết)

Giảng viên Học phần
Khoa Công nghệ Thông tin

Dành cho sinh viên Khóa 2026 – Lớp IT26M

Dựa trên giáo trình: *Building Java Programs* (Stuart Reges & Marty Stepp)

Năm học 2026

Mục tiêu học tập bài học (Course Learning Outcomes – CLOs)

Chuẩn đầu ra sau khi hoàn thành Chương 1 (Buổi học 3 tiết):

- ✓ **CLO 1.1 (Hiểu):** Giải thích bản chất kiến trúc máy tính, hệ nhị phân, cơ chế biên dịch vs. thông dịch, vai trò của **máy ảo JVM** và mã **Bytecode** (WORA).
- ✓ **CLO 1.2 (Áp dụng):** Viết thành thạo khung chương trình Java cơ bản (`class`, `main`), phân biệt `print` và `println`, sử dụng chính xác các **ký tự thoát** (`\n, \t, \", \\`).
- ✓ **CLO 1.3 (Phân tích):** Nhận diện và phân loại chính xác **3 loại lỗi** (Syntax, Runtime, Logic); đọc thông báo lỗi từ trình biên dịch để gỡ lỗi (debug).
- ✓ **CLO 1.4 (Thiết kế):** Vận dụng tư duy **Phân rã thủ tục (Procedural Decomposition)** và nguyên lý **DRY**; khai báo và gọi phương thức tĩnh (`static methods`); phân tích luồng điều khiển (*Flow of Control*).
- ✓ **CLO 1.5 (Sáng tạo):** Độc lập viết, biên dịch qua CLI (`javac/java`) và IDE một chương trình hoàn chỉnh vẽ hình nghệ thuật ASCII có cấu trúc nhiều cấp.

Lộ trình buổi học 3 tiết (Tích hợp Lý thuyết & Thực hành)

TIẾT 1 (50 phút)

Nền tảng & Khởi đầu:

- Khái niệm tính toán & Nhị phân
- Compiler vs. Interpreter
- Kiến trúc JVM & Bytecode
- Quy trình: Edit-Compile-Run
- Chương trình `Hello.java`
- Lệnh `print` vs `println`

[Lab 1]: Viết & chạy Java trên CLI và IDE.

TIẾT 2 (50 phút)

Cú pháp & Gỡ lỗi:

- Ký tự thoát (*Escape Seq*)
- Định danh, Từ khóa, CamelCase
- Quy ước chú thích (Comments)
- Phân loại 3 nhóm lỗi:
  - Lỗi cú pháp (Syntax)
  - Lỗi thực thi (Runtime)
  - Lỗi ngữ nghĩa (Logic)
- Chiến lược đọc lỗi & gỡ lỗi

[Lab 2]: Thử thách Debug & Escape Strings.

TIẾT 3 (50 phút)

Phân rã & Case Study:

- Tư duy phân rã Top-down
- Phương thức tĩnh (`static`)
- Luồng điều khiển (*Control Flow*)
- Mục tiêu: Structure & DRY
- Case Study `DrawFigures`:
  - Bản 1: Naïve (đơn khối)
  - Bản 2: Phân rã cấp 1
  - Bản 3: Phân rã triệt để

TIẾT 1

NỀN TẢNG HỆ THỐNG TÍNH TOÁN, KIẾN TRÚC MÁY ẢO JVM & XÂY DỰNG CHƯƠNG TRÌNH JAVA ĐẦU TIÊN

Thời lượng: 50 phút | 35’ Lý thuyết tương tác + 15’ Hands-on Lab 1

1.1 Tại sao chúng ta cần học lập trình? (Why Programming?)

Bản chất của khoa học máy tính

> *"Người ta thường nói rằng một người không thực sự hiểu điều gì đó cho đến khi họ dạy nó cho người khác. Trên thực tế, một người **không thực sự hiểu** điều gì đó cho đến khi họ **dạy nó cho một chiếc máy tính**, tức là diễn đạt nó dưới dạng một thuật toán."*
>
> *— Donald Knuth (Turing Award, Cha đẻ ngành Phân tích thuật toán)*

- **Máy tính:** Thiết bị phần cứng vạn năng (General-purpose device).
- **Chương trình (Program):** Tập hợp các chỉ thị ra lệnh cho phần cứng.
- **Lập trình:** Rèn luyện tư duy giải quyết vấn đề có

Thành phần cốt lõi của máy tính

- **CPU (Bộ vi xử lý trung tâm):** "Bộ não" thực thi các phép tính logic và số học.
- **Bộ nhớ chính (RAM):** Lưu trữ tạm thời, tốc độ cực nhanh, mất dữ liệu khi mất nguồn.
- **Bộ nhớ thứ cấp (Ổ cứng/SSD):** Lưu trữ dữ liệu vĩnh viễn (persistent storage).
- **Thiết bị I/O:** Bàn phím, chuột, màn hình, kết nối mạng.

Thế giới số & Biểu diễn dữ liệu bằng Hệ nhị phân (Binary)

Tại sao máy tính sử dụng số nhị phân?

- Phần cứng vật lý cấu tạo từ hàng tỷ bóng bán dẫn (*transistors*).
- Hai trạng thái vật lý ổn định nhất:
  - **Bật (On) / Điện áp cao:** Đại diện cho số **1**.
  - **Tắt (Off) / Điện áp thấp:** Đại diện cho số **0**.
- **Bit (Binary Digit):** Đơn vị thông tin nhỏ nhất (0 hoặc 1).
- **Byte:** Nhóm 8 bits. 1 Byte biểu diễn được $2^8 = 256$ trạng thái (từ 0 đến 255).

Cơ chế biểu diễn số theo cơ số 2

So sánh hệ thập phân (Base-10) và nhị phân (Base-2):

Trọng số	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$
Bit	**1**	**0**	**1**	**1**

$$(1011)_2 = 1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1$$
$$= 8 + 0 + 2 + 1 = \mathbf{11}_{10}$$

$\rightarrow$ Mọi chương trình, hình ảnh, video, âm thanh cuối cùng đều được ánh xạ về các dãy bit 0 và 1!

Cơ chế thực thi: Trình biên dịch (Compiler) vs. Thông dịch (Interpreter)

Trình biên dịch truyền thống (C/C++)

- Dịch toàn bộ mã nguồn (*Source code*) thành **Mã máy trực tiếp (Native Machine Code)**.
- **Ưu điểm:** Tốc độ thực thi cực nhanh, tối ưu phần cứng.
- **Nhược điểm chí tử:** **Phụ thuộc nền tảng!** File thực thi trên Intel x86 Windows không thể chạy trên ARM macOS hay Linux.

Trình thông dịch (Python, JavaScript)

- Đọc và diễn giải trực tiếp từng dòng lệnh tại thời điểm chạy.
- **Ưu điểm:** Linh hoạt, dễ thử nghiệm trực tiếp.
- **Nhược điểm:** Tốc độ thực thi chậm hơn nhiều so với mã đã biên dịch sẵn.

Câu hỏi đặt ra cho các nhà thiết kế ngôn ngữ Java (Năm 1995)

Làm sao vừa đạt được tốc độ thực thi cao, vừa có thể chạy trên **mọi nền tảng phần cứng** mà không cần biên dịch lại?

Kiến trúc đột phá của Java: Máy ảo JVM & Mã Bytecode

Triết lý: "Write Once, Run Anywhere" (WORA)

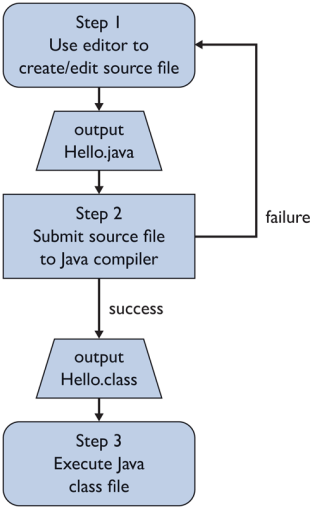
- Java kết hợp cả **Biên dịch** lẫn **Máy ảo**!
- Trình biên dịch Java (`javac`) không dịch ra mã máy thật của CPU.
- Nó dịch mã nguồn (`.java`) thành **Java Bytecode** (`.class`).
- Bytecode là tập lệnh chuẩn cho một cỗ máy lý thuyết gọi là **Java Virtual Machine (JVM)**.
- Mỗi hệ điều hành (Windows, Linux, macOS) có một bản JVM riêng biệt để thông dịch/JIT-compile Bytecode ra mã máy phần cứng.

Bộ công cụ Java (Phân biệt JDK, JRE, JVM)

- **JVM (Java Virtual Machine):** Môi trường thực thi Bytecode.
- **JRE (Java Runtime Environment):** Gồm JVM + Thư viện chuẩn (*Java Class Libraries*) để người dùng chạy ứng dụng.
- **JDK (Java Development Kit):** Dành cho lập trình viên! Chứa JRE + Trình biên dịch (`javac`) + Công cụ gỡ lỗi.

JDK = JRE + Tools ⊃
JRE = JVM + Libs

Quy trình phát triển chương trình Java: Edit → Compile → Run



Hình 1.1: Quá trình tạo và thực thi chương trình Java (Trích giáo trình Chapter 1, p.21)

3 Bước cốt lõi của mọi lập trình viên Java:

1. **Bước 1: Soạn thảo (Edit)**
   - Viết mã nguồn vào file văn bản thuần.
   - Bắt buộc đuôi file: `Hello.java`.
2. **Bước 2: Biên dịch (Compile)**
   - Lệnh: `javac Hello.java`
   - Kiểm tra lỗi cú pháp.
   - Tạo ra file bytecode: `Hello.class`.
3. **Bước 3: Thực thi (Run)**
   - Khởi động máy ảo JVM: `java Hello`
   - JVM nạp bytecode và chuyển cho CPU thực thi.
   - Kết quả xuất hiện trên màn hình!

1.2 Chương trình Java hoàn chỉnh đầu tiên

Listing 1: File: Hello.java

```java
// Chuong trinh Java dau tien cua lop IT26M
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, world!");
    }
}

```

Quy tắc vàng bắt buộc:

- Mọi mã nguồn Java đều phải nằm **bên trong một lớp (class)**.
- Tên file **bắt buộc phải trùng khớp 100%** với tên lớp công khai:

  Lớp `public class Hello` $\Longleftrightarrow$ File `Hello.java`
- Java phân biệt chữ hoa và chữ thường (*case-sensitive*).

Cấu trúc mẫu (Syntax Template)

Khai báo lớp chuẩn:

```
public class <TênLớp> {
... các phương thức ...
}
```

Cặp ngoặc nhọn { } :
Đóng gói phạm vi (*scope*) của khối lệnh. Mọi ngoặc mở { đều phải có ngoặc đóng } tương ứng.

Mổ xẻ phương thức `main` – Điểm bắt đầu của mọi ứng dụng

Dòng lệnh then chốt:

```
public static void main(String[] args) { ... }
```

Từ khóa	Ý nghĩa học thuật	Diễn giải dành cho sinh viên mới học
`public`	Quyền truy cập công khai	Bất kỳ đối tượng nào (bao gồm máy ảo JVM) cũng có thể gọi phương thức này từ bên ngoài.
`static`	Phương thức tĩnh cấp lớp	Có thể thực thi trực tiếp từ lớp mà không cần khởi tạo đối tượng (*instance*).
`void`	Kiểu dữ liệu trả về	Phương thức này thực hiện công việc và **không trả lại** giá trị nào.
`main`	Tên phương thức	Tên quy ước đặc biệt mà JVM tìm kiếm đầu tiên khi chạy chương trình.
`String[] args`	Mảng tham số dòng lệnh	Cho phép truyền các chuỗi đối số từ bên ngoài khi khởi động chương trình.

Lệnh xuất dữ liệu: `println` vs. `print`

System.out.println(...)

In nội dung ra console và **tự động xuống dòng mới** (newline).

```
1 System.out.println("Dong thu nhat");
2 System.out.println("Dong thu hai");
3
```

Kết quả trên màn hình:

```
Dong thu nhat
Dong thu hai
```

System.out.print(...)

In nội dung ra console và **giữ nguyên con trỏ** ở cuối dòng hiện tại.

```
1 System.out.print("Xin ");
2 System.out.print("chao ");
3 System.out.println("IT26M!");
4
```

Kết quả trên màn hình:

```
Xin chao IT26M!
```

Lưu ý quan trọng:

Lệnh `System.out.println();` (không tham số) dùng để tạo ra một dòng trống trên console. Mọi câu lệnh trong Java **bắt buộc kết thúc bằng dấu chấm phẩy ;**!

[CHECKPOINT 1] Kiểm tra nhanh lý thuyết Tiết 1

Câu hỏi 1: Lệnh nào hợp lệ trong Java?

A. `system.out.println("Hello");`
B. `System.Out.println("Hello");`
C. `System.out.println("Hello");`
D. `System.out.println(Hello);`

Câu hỏi 2: Vai trò của file có đuôi `.class`?

A. Chứa mã nguồn viết bằng tiếng Anh cho lập trình viên đọc.
B. Chứa mã Bytecode đã được biên dịch để máy ảo JVM thực thi.
C. Là file mã máy nhị phân chạy trực tiếp trên chip Intel.
D. Là file cấu hình môi trường của IDE.

Giải thích đáp án

Đáp án Câu 1: C
Java phân biệt hoa thường: `System` viết hoa chữ S, `out` và `println` viết thường. Chuỗi chữ phải nằm trong cặp nháy kép " ".

Đáp án Câu 2: B
Trình biên dịch `javac` sinh ra file `.class` chứa Bytecode độc lập nền tảng cho JVM.

[HANDS-ON LAB 1] Thực hành tại chỗ: Viết, biên dịch & chạy Java

Nhiệm vụ thực hành Tiết 1 (Thời gian: 15 phút – Tất cả sinh viên thực hiện)

Yêu cầu 1: Thao tác Command Line (Terminal / CMD / PowerShell)

1. Mở Terminal, gõ `javac -version` và `java -version` để kiểm tra JDK.
2. Dùng trình soạn thảo (Notepad / VS Code) tạo thư mục `Lab1` và file `WelcomeIT26M.java`.
3. Biên dịch bằng lệnh CLI: `javac WelcomeIT26M.java` → Dùng `dir` hoặc `ls` quan sát file `WelcomeIT26M.class` vừa xuất hiện!
4. Chạy chương trình: `java WelcomeIT26M`

Listing 2: Mã nguồn WelcomeIT26M.java

```
1  public class WelcomeIT26M {
2      public static void main(String[] args) {
3          System.out.println("Chao mung lop IT26M den voi
           Java!");
4          System.out.println("Viet mot lan, chay moi noi
           (WORA).");
5      }
6  }
```

Output hiển thị chính xác:

```
Chao mung lop IT26M den voi Java!
Viet mot lan, chay moi noi (WORA).
```